

Implementación de Autenticación Multifactor (MFA) en Entornos Reales

Jhoan Sebastian Parra
David Burbano Mariño

Seguridad Informática

Luz Marina Santos



Universidad de Pamplona
Junio 24 de 2026

Índice

1	Título y Descripción General	2
1.1	Título del Proyecto	2
1.2	Descripción General	2
2	Problema Identificado y Justificación del Uso de MFA	2
2.1	Problema	2
2.2	Justificación	3
3	Objetivo General y Objetivos Específicos	3
3.1	Objetivo General	3
3.2	Objetivos Específicos	3
4	Diagrama de Arquitectura Propuesta	4
4.1	Componentes del Sistema	4
4.2	Flujo de Datos: Inicio de Sesión con 2FA	5
4.3	Flujo de Datos: Configuración de 2FA	6
5	Herramientas Tecnológicas Seleccionadas	6
5.1	Criterios de Selección	8
6	Evaluación de Resultados	8
6.1	Cómo MFA Mejora la Seguridad	8
6.2	Limitaciones de la Implementación Actual	9
6.3	Propuestas de Mejora Futura	9
6.4	Desafíos de Ingeniería Encontrados	10
7	Conclusiones Generales	11

Título y Descripción General

Título del Proyecto

Implementación de Autenticación Multifactor (MFA) en el Simulador de Inversiones FIUP.

Descripción General

El Simulador de Inversiones FIUP es una plataforma web educativa desarrollada para la Universidad de Pamplona. Permite a los estudiantes gestionar portafolios bursátiles virtuales, consultar cotizaciones en tiempo real mediante la API de Finnhub y participar en rankings académicos. La plataforma maneja datos personales (correo electrónico, número telefónico) y un saldo virtual de hasta 10,000 USD, lo que la convierte en un objetivo atractivo para ataques de suplantación de identidad.

El proyecto consistió en integrar un sistema de Autenticación Multifactor (MFA) sobre la arquitectura existente del simulador. La implementación se realizó sobre un backend en FastAPI con Python 3.12 y un frontend en React 18 con TypeScript. El MFA soporta tres modalidades de segundo factor: códigos TOTP generados por aplicaciones como Google Authenticator, códigos SMS enviados a través de una puerta de enlace HTTP, y códigos de respaldo únicos pregenerados.

El flujo de inicio de sesión se divide en dos pasos. Primero, el usuario ingresa su nombre de usuario y contraseña. Si las credenciales son válidas y el usuario tiene habilitado el 2FA, el servidor emite un token temporal con validez de 30 segundos. En el segundo paso, el usuario debe presentar un código TOTP de 6 dígitos, un código SMS o uno de sus códigos de respaldo. Solo entonces se emiten los tokens de acceso y actualización definitivos.

Problema Identificado y Justificación del Uso de MFA

Problema

Las aplicaciones web financieras son el blanco principal de ataques de robo de credenciales. Un estudio del Verizon Data Breach Investigations Report de 2025 señala que el 74% de las brechas en aplicaciones web involucran credenciales débiles, robadas o reutilizadas. El Simulador FIUP, aunque es un entorno educativo, no está exento de estos riesgos: almacena datos personales de estudiantes y gestiona activos virtuales que los usuarios valoran.

El problema central es que una contraseña, incluso con alta entropía, es un único

punto de fallo. Si un atacante obtiene la contraseña mediante phishing, keylogging, credential stuffing o una filtración de base de datos, obtiene acceso total a la cuenta de la víctima. En el contexto del simulador, esto permite:

- Manipular portafolios ajenos (compra y venta de activos).
- Robar datos personales de perfil (correo, teléfono).
- Suplantar la identidad del usuario en el ranking académico.
- Escalar a ataques contra otros usuarios mediante ingeniería social.

Justificación

MFA mitiga este problema porque introduce un segundo factor que el atacante no posee. El modelo de seguridad cambia de “algo que sabes” (la contraseña) a “algo que sabes + algo que tienes”. Incluso si la contraseña se ve comprometida, el atacante necesita acceso físico al dispositivo del usuario para generar el código TOTP, o haber interceptado el mensaje SMS correspondiente.

Seleccionamos TOTP como factor principal porque es un estándar abierto (RFC 6238), no requiere conectividad del usuario (el código se genera localmente en el dispositivo) y no depende de operadores de telefonía. Los códigos SMS los incorporamos como factor de contingencia para usuarios sin acceso a un smartphone con aplicación autenticadora. Los códigos de respaldo, almacenados como hash SHA-256, cubren el escenario de pérdida del dispositivo.

La implementación no se limita a agregar un segundo factor. Incluye medidas complementarias: rate limiting de 5 intentos por minuto en el endpoint de inicio de sesión, tokens JWT con expiración corta (15 minutos para acceso, 7 días para actualización), revocación de todos los tokens al cambiar la contraseña mediante un contador de versión, y cookies HTTPOnly con SameSite=Lax para mitigar ataques XSS y CSRF.

Objetivo General y Objetivos Específicos

Objetivo General

Implementar un sistema de autenticación multifactor funcional en el Simulador de Inversiones FIUP que combine contraseña y códigos TOTP (RFC 6238) para reducir el riesgo de accesos no autorizados por robo de credenciales.

Objetivos Específicos

- 01** Diseñar e implementar un flujo de registro con verificación de correo electrónico mediante código OTP de 6 dígitos, almacenado en Redis con expiración de 10

minutos y un máximo de 3 intentos de validación.

- 02** Desarrollar un módulo de generación y verificación de códigos TOTP usando la librería `pyotp`, incluyendo la generación de códigos QR para facilitar la configuración en aplicaciones autenticadoras.
- 03** Implementar un sistema de tokens JWT con dos niveles: un token temporal de 30 segundos para el desafío 2FA y tokens de acceso/actualización con claims de seguridad (`aud`, `iss`, `jti`, `type`, `password_version`).
- 04** Integrar rate limiting granular en todos los endpoints sensibles (login, registro, verificación 2FA, envío de SMS) usando `slowapi` con identificación por dirección IP.
- 05** Generar y almacenar 8 códigos de respaldo por usuario, cifrados con SHA-256 y de un solo uso, para recuperación de cuenta en caso de pérdida del dispositivo.
- 06** Construir una interfaz de usuario en React que guíe al usuario a través de la configuración y el uso del 2FA, con manejo de estados de carga, error y expiración de token temporal.

Diagrama de Arquitectura Propuesta

Componentes del Sistema

El sistema sigue una arquitectura de tres capas con un proxy inverso Nginx al frente. La siguiente tabla describe los componentes y su rol en el flujo MFA:

Cuadro 1: Componentes de la arquitectura MFA

Componente	Rol en MFA
Nginx (proxy)	Terminación SSL, rate limiting adicional, cabeceras de seguridad (HSTS, CSP, X-Frame-Options)
FastAPI (backend)	Orquestación del flujo MFA: validación de credenciales, emisión de tokens, verificación TOTP, envío de OTP
PostgreSQL	Almacenamiento persistente: usuarios, secretos TOTP, códigos de respaldo hasheados, códigos de verificación
Redis	Almacenamiento temporal: datos de registro, tokens temporales de 2FA, contadores de rate limiting
Resend API	Entrega de códigos OTP por correo electrónico durante el registro
SMS Gateway	Entrega de códigos OTP por mensaje de texto durante el inicio de sesión
React SPA (frontend)	Interfaz de usuario con manejo de estados 2FA, visualización de códigos QR, entrada de códigos

Flujo de Datos: Inicio de Sesión con 2FA

A continuación se describe el flujo paso a paso que debe reflejarse en el diagrama de arquitectura:

Paso 1: Credenciales. El usuario completa el formulario de inicio de sesión con su nombre de usuario y contraseña. El frontend envía una petición POST a `/api/v1/login`.

Paso 2: Validación. El backend consulta la tabla `users` en PostgreSQL. Verifica la contraseña con `bcrypt` (12 rondas). Si el usuario tiene `totp_enabled = true`, no emite los tokens de acceso todavía. En su lugar, genera un token JWT temporal con los siguientes atributos: tipo `temp`, expiración de 30 segundos, y un identificador único `jti`. Almacena el `jti` en Redis para garantizar que el token solo pueda usarse una vez. La respuesta incluye `{requires_2fa: true, temp_token, available_methods}`.

Paso 3: Selección de método. El frontend inspecciona `available_methods` y muestra las opciones disponibles (TOTP, SMS, código de respaldo). Si solo hay un método, avanza directamente.

Paso 4: Verificación TOTP. El usuario ingresa el código de 6 dígitos de su aplicación autenticadora. El frontend envía una petición POST a `/api/v1/2fa/login-verify` con `{temp_token, code}`.

Paso 5: Validación del segundo factor. El backend decodifica el `temp_token` y verifica que el `jti` exista en Redis y no haya sido usado. Luego lee el `totp_secret`

del usuario desde PostgreSQL y ejecuta `pyotp.TOTP(secret).verify(code, valid_window=)`. Si el código es válido, elimina el `jti` de Redis (consumo del token temporal). Genera un `access_token` con expiración de 15 minutos y un `refresh_token` con expiración de 7 días. Ambos incluyen el `password_version` actual del usuario.

Paso 6: Entrega de sesión. El backend establece cookies HTTPOnly con los tokens y devuelve el objeto de usuario en el cuerpo de la respuesta. El frontend almacena el estado de autenticación en Zustand (solo el objeto de usuario, nunca los tokens) y redirige al tablero principal.

Flujo de Datos: Configuración de 2FA

Paso 1: Solicitud de configuración. El usuario autenticado accede a la página de configuración y hace clic en “Habilitar 2FA”. El frontend llama a `GET /api/v1/2fa/setup`.

Paso 2: Generación de secreto. El backend genera un secreto TOTP de 16 caracteres en base32 usando `pyotp.random_base32()`. Construye la URI de aprovisionamiento según el formato `otpauth://totp/...`. Genera un código QR en formato PNG y lo codifica en base64. Devuelve `{secret, qr_code, provisioning_uri}`. El secreto aún no se persiste en la base de datos.

Paso 3: Escaneo y verificación. El usuario escanea el código QR con su aplicación autenticadora (Google Authenticator, Authy, etc.) e ingresa el código de 6 dígitos que la aplicación genera. El frontend envía `POST /api/v1/2fa/verify {code}`.

Paso 4: Activación. El backend verifica el código. Si es válido, persiste el secreto en la columna `totp_secret` del usuario en PostgreSQL, establece `totp_enabled = true` y genera 8 códigos de respaldo. Cada código de respaldo tiene el formato `XXXXXX-XXXXXX` y se almacena como hash SHA-256 en la tabla `backup_codes`. La respuesta incluye los códigos de respaldo en texto plano para que el usuario los guarde.

Herramientas Tecnológicas Seleccionadas

La siguiente tabla resume las herramientas utilizadas en la implementación del MFA, junto con su propósito específico dentro del flujo de autenticación.

Cuadro 2: Herramientas tecnológicas del proyecto MFA

Herramienta	Versión	Propósito en MFA
Python 3.12 + FastAPI 0.135		Backend asíncrono del flujo MFA. Maneja solicitudes concurrentes sin bloquear el event loop durante verificación de OTP o consultas a Redis
bcrypt	5.0+	Hashing de contraseñas con 12 rondas. Resistente a ataques con GPU y ASIC. Supera el mínimo recomendado por NIST de 10 rondas
PyJWT	2.12	Creación y verificación de tokens JWT. Soporta claims personalizados (aud, iss, jti, type) para prevenir reúsos entre servicios
pyotp	2.9	Implementación del estándar TOTP RFC 6238. Generación de secretos, códigos QR y verificación con ventana de tolerancia
qrcode[pil]	7.4	Generación de códigos QR en servidor para facilitar la configuración del 2FA sin transferir secretos por canales inseguros
slowapi	0.1.9	Rate limiting por dirección IP. Límites diferenciados: 5/min en login, 3/min en envío de SMS, 10/min en verificación 2FA
Redis	7.0	Almacenamiento en memoria de datos efímeros: tokens temporales de 2FA, datos de registro, contadores de rate limiting. TTL automático
Resend	0.5	API de envío de correos electrónicos para códigos OTP de verificación de registro
httpx	0.28	Cliente HTTP asíncrono para la comunicación con la puerta de enlace SMS externa
React 18 + TypeScript 18/5.3		Interfaz de usuario con manejo de estados 2FA, visualización de códigos QR y entrada de códigos con validación Zod
Zustand	4	Gestión de estado del lado del cliente. Persiste solo el objeto de usuario en localStorage, nunca los tokens
Zod	–	Validación de esquemas en el frontend: formato de códigos, complejidad de contraseñas, campos obligatorios

Criterios de Selección

Elegimos estas herramientas por cuatro razones principales. Primera, la compatibilidad con el stack existente del simulador (FastAPI + React), lo que evitó reescribir la infraestructura. Segunda, la madurez de las bibliotecas: pyotp tiene más de 10 años de desarrollo continuo y es la implementación Python de referencia para TOTP. Tercera, el rendimiento asíncrono: tanto FastAPI como httpx y redis-py operan sobre el mismo event loop, eliminando cuellos de botella en la verificación de códigos. Cuarta, el costo: todas las herramientas son de código abierto y no requieren licencias adicionales.

Evaluación de Resultados

Cómo MFA Mejora la Seguridad

La tabla siguiente compara el estado de seguridad del sistema antes y después de la implementación del MFA:

Cuadro 3: Evaluación de mejora de seguridad con MFA

Métrica	Sin MFA	Con MFA	Impacto
Factores de autenticación	1 (contraseña)	2 (contraseña + TOTP/SMS/respaldo)	El atacante necesita ambos factores
Contraseña comprometida → acceso total	Sí	No	Reduce ventana de ataque
Fuerza bruta en login	Sin límite efectivo	5 intentos/min por IP + segundo paso	Requiere múltiples IPs para mantener 1 intento/s
Token interceptado válido por	30 minutos	15 min + <i>password_version</i> 50%	Ventana de ataque reducida
Registro de cuentas falsas	Sin verificación	OTP obligatorio por email	Previene creación masiva automatizada
Sesión secuestrada por XSS	Posible (token en localStorage)	Cookies HTTPOnly + SameSite=Lax	Token inaccesible desde JavaScript

El mayor impacto se da en el escenario de credential stuffing. Un atacante que obtiene una lista de contraseñas filtradas no puede iniciar sesión sin tener además

el dispositivo del usuario. Incluso si logra evadir el rate limit usando una botnet, debe resolver el desafío TOTP, que expira en 30 segundos y es de un solo uso.

Limitaciones de la Implementación Actual

Ningún sistema de seguridad es perfecto. Identificamos varias limitaciones que deben considerarse:

- **Secreto TOTP en texto plano.** El campo *totp_secret* en la base de datos PostgreSQL no está cifrado. Si un atacante obtiene un volcado de la base de datos, mediante inyección SQL, acceso físico o una copia de seguridad mal protegida, puede generar códigos TOTP válidos sin límite para cualquier usuario. La solución pasa por cifrar el secreto con Fernet (cryptography simétrica) usando una clave derivada del *SECRET_KEY* de la aplicación.
- **El cierre de sesión no invalida los tokens.** La función de logout actualmente solo elimina las cookies del navegador. Los tokens JWT subyacentes siguen siendo válidos hasta su expiración natural (15 minutos para access token, 7 días para refresh token). Un atacante que haya interceptado un token puede seguir usándolo incluso después de que el usuario cierre la sesión. La solución es mantener una lista negra de tokens en Redis (*blacklist:{jti}*) y verificar contra ella en cada solicitud.
- **Ausencia de CAPTCHA.** El rate limiting por IP es efectivo contra atacantes individuales, pero una botnet distribuida puede eludirlo fácilmente usando miles de direcciones IP diferentes. Agregar Google reCAPTCHA v3 o hCaptcha en los formularios de login y registro añadiría una capa de defensa basada en análisis de comportamiento.
- **Dependencia de SMS como factor.** La NIST ya no recomienda SMS como segundo factor desde la publicación del SP 800-63 en 2017. Los códigos SMS son vulnerables a ataques de SIM swapping y a la interceptación de la señal SS7. En nuestro diseño lo mantenemos como factor de contingencia, no como factor principal, pero la recomendación es migrar gradualmente a WebAuthn (FIDO2/Passkeys).
- **Sin registro de auditoría.** El sistema no genera registros de eventos 2FA: quién habilitó la autenticación, desde qué dirección IP, cuándo se usó un código de respaldo, o cuántos intentos fallidos de 2FA han ocurrido. Esta información es necesaria para el análisis forense en caso de incidente.

Propuestas de Mejora Futura

Las limitaciones anteriores se traducen en un plan de mejoras organizado en tres horizontes temporales:

Corto plazo (próximos 30 días):

- Cifrar el secreto TOTP con la biblioteca *cryptography.fernet*. La clave de cifrado se deriva del *SECRET_KEY* de la aplicación usando HKDF.
- Implementar una lista negra de tokens JWT en Redis. Al cerrar sesión, el *jwt* del token se agrega a la lista negra con un TTL igual al tiempo restante de expiración del token. Cada solicitud entrante verifica que su *jwt* no esté en la lista negra.
- Agregar logging estructurado para eventos de 2FA: habilitación, deshabilitación, inicio de sesión exitoso y uso de código de respaldo.

Mediano plazo (1 a 3 meses):

- Integrar Google reCAPTCHA v3 en los formularios de inicio de sesión y registro. reCAPTCHA v3 funciona en segundo plano sin interrumpir al usuario, asignando una puntuación de riesgo que el backend puede usar para denegar solicitudes sospechosas.
- Implementar WebAuthn (FIDO2/Passkeys) como factor adicional. A diferencia del TOTP, WebAuthn es resistente a ataques de phishing porque vincula la credencial al origen del sitio web.
- Agregar un panel de actividad de cuenta donde el usuario pueda ver los últimos inicios de sesión con dirección IP, dispositivo y método 2FA utilizado.

Largo plazo (próximo ciclo académico):

- Deprecar el SMS como factor 2FA y reemplazarlo completamente por notificaciones push a una aplicación móvil dedicada.
- Implementar un sistema de detección de anomalías que analice patrones de inicio de sesión (ubicación geográfica, horario, agente de usuario) y solicite autenticación adicional cuando el riesgo sea alto.

Desafíos de Ingeniería Encontrados

Durante el desarrollo enfrentamos problemas que vale la pena documentar. El manejo de la expiración del token temporal fue el más complejo: si el usuario se demora más de 30 segundos en ingresar el código TOTP, el token expira y debe comenzar de nuevo. En la interfaz de usuario, esto se traduce en un mensaje de “Sesión expirada” y un redireccionamiento al paso de credenciales. Al principio no manejábamos este caso y el usuario veía un error genérico.

Otro desafío fue la ventana de tolerancia de reloj. El estándar TOTP asume que el reloj del dispositivo del usuario está sincronizado con el servidor. En pruebas encontramos que algunos teléfonos tenían una diferencia de hasta 45 segundos, lo que provocaba falsos rechazos. Solucionamos esto configurando *valid_window=1* en pyotp, que acepta el código del intervalo anterior y el siguiente además del

actual. Esto amplía la ventana a 90 segundos en total, un compromiso aceptable entre seguridad y usabilidad.

La gestión de sesiones comprometidas requirió el mecanismo de *password_version*. Cada vez que el usuario cambia su contraseña, este contador se incrementa y todos los tokens emitidos con la versión anterior se invalidan automáticamente. Implementamos esto en *auth_service.py* y lo verificamos en cada solicitud protegida.

Conclusiones Generales

La implementación de MFA en el Simulador de Inversiones FIUP demuestra que es posible integrar autenticación multifactor en una aplicación web existente sin reescribir la arquitectura base. El sistema soporta tres modalidades de segundo factor (TOTP, SMS y códigos de respaldo), lo que cubre tanto el escenario principal como las situaciones de contingencia.

Desde el punto de vista técnico, la implementación cumple con los estándares de seguridad establecidos: bcrypt con 12 rondas para contraseñas, JWT con validación de audiencia y emisor, tokens temporales de un solo uso, rate limiting granular y revocación de sesión por cambio de contraseña. El uso de Redis como almacenamiento de datos efímeros permitió manejar los tokens temporales y los códigos OTP con tiempos de expiración precisos sin contaminar la base de datos relacional.

La evaluación de resultados muestra una mejora significativa en la postura de seguridad. Un atacante que comprometa la contraseña de un usuario todavía necesita acceso al dispositivo físico para completar la autenticación. El rate limiting reduce la tasa de ataques de fuerza bruta a 5 intentos por minuto por dirección IP. La combinación de cookies HTTPOnly, SameSite=Lax y la ausencia de tokens en localStorage elimina vectores de ataque XSS y CSRF.

Sin embargo, el sistema tiene limitaciones que no deben ignorarse. La más crítica es el almacenamiento del secreto TOTP en texto plano en la base de datos. Le sigue la falta de invalidación de tokens al cerrar sesión. Ambas son corregibles en el corto plazo y no invalidan el prototipo funcional. La ausencia de CAPTCHA y de registro de auditoría son deficiencias que deben abordarse antes de un despliegue en producción.

Como trabajo futuro, la migración a WebAuthn (FIDO2/Passkeys) representa el siguiente paso lógico. Este estándar elimina la dependencia de códigos de 6 dígitos y ofrece resistencia al phishing, algo que el TOTP no puede garantizar. La integración de un sistema de detección de anomalías basado en patrones de inicio de sesión permitiría escalar dinámicamente los requisitos de autenticación según el nivel de riesgo.

En conclusión, el prototipo de MFA implementado es funcional, sigue buenas prácticas de seguridad y cumple con los objetivos académicos planteados. Con las correcciones señaladas, constituye una base sólida para un sistema de autenticación robusto en un entorno educativo y, con mejoras adicionales, para un despliegue en producción.